

CSC3066

Deep Learning

Perceptron

Anna Jurek-Loughrey



What's to come today

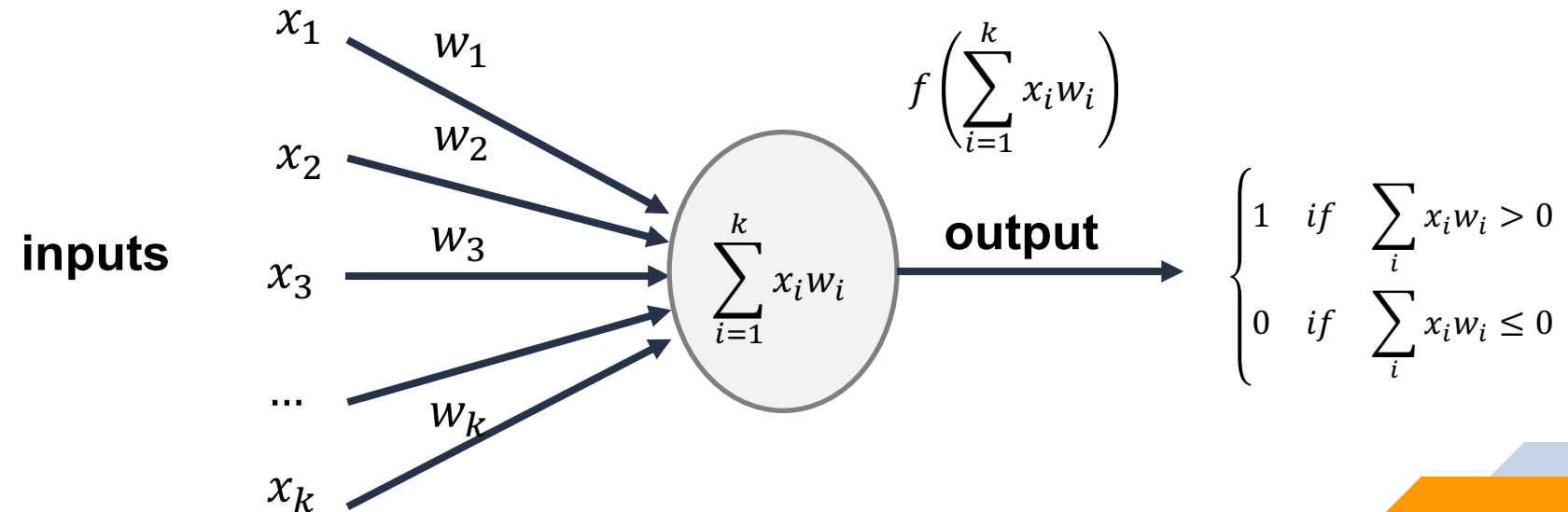
- ❑ Perceptron Model
- ❑ Simple Update Rule for determining optimal parameters of perceptron model

Perceptron



Perceptron: Definition

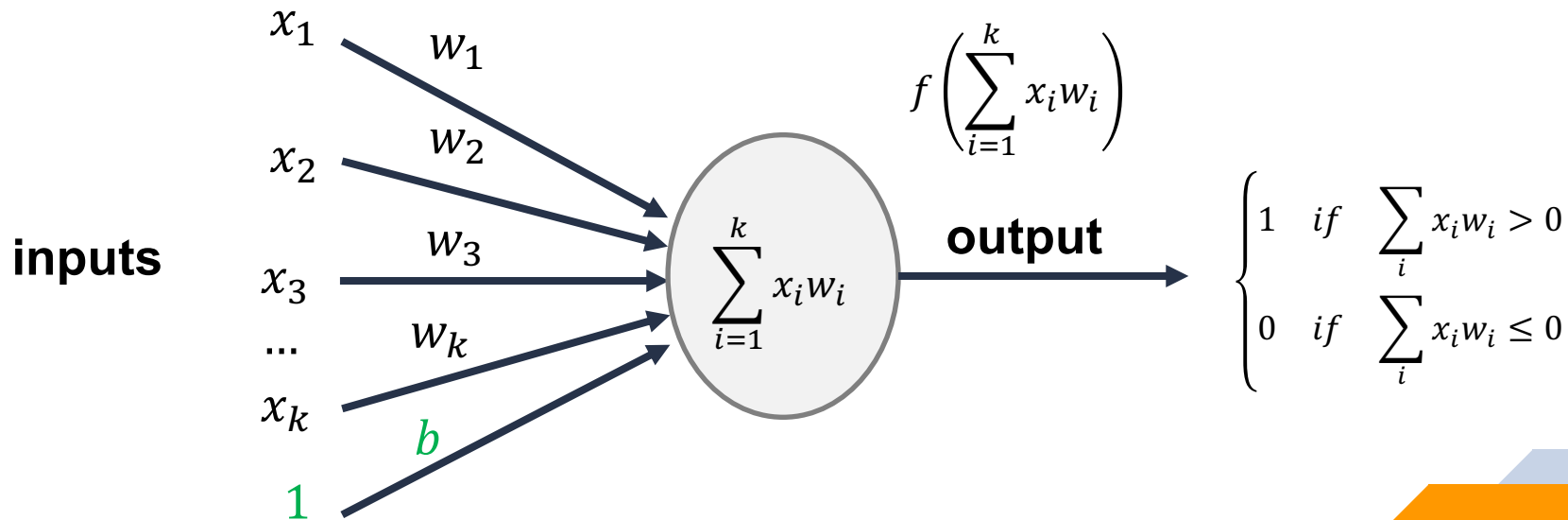
- **Perceptron** is the simplest Neural Network. It is composed of only a single neuron.





Perceptron: Definition

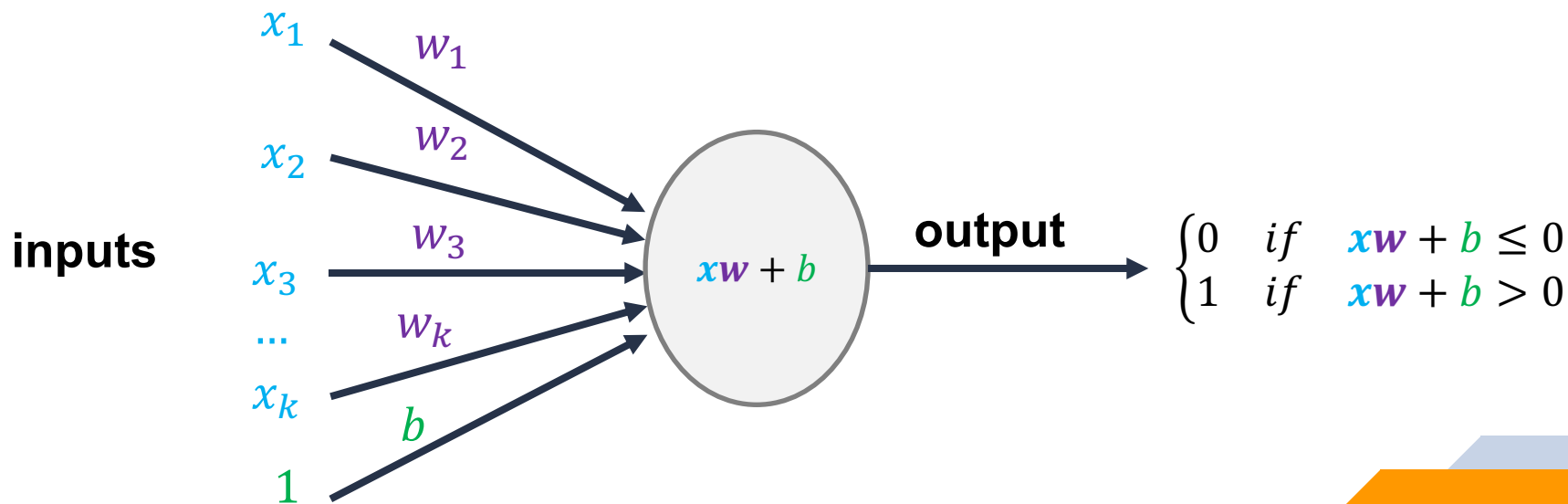
- A constant (**bias**) can be added to the product of features and weights to offset the activation function.





Perceptron: Definition

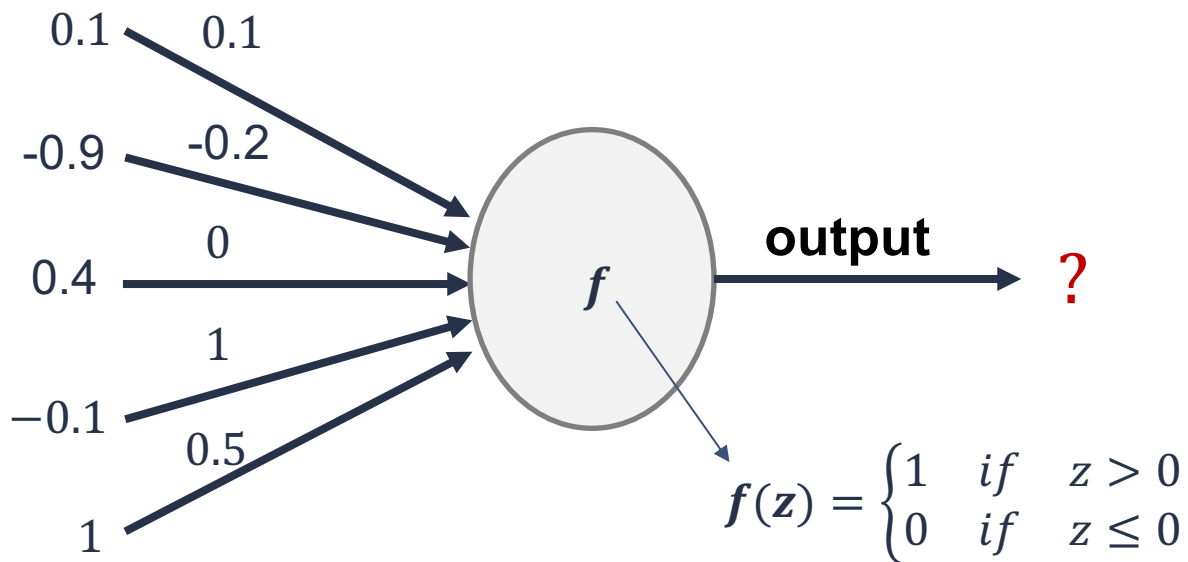
■ We can also write it in vector notation: $x = [x_1, \dots, x_k]$ $w = \begin{bmatrix} w_1 \\ \dots \\ w_k \end{bmatrix}$





Perceptron: Definition

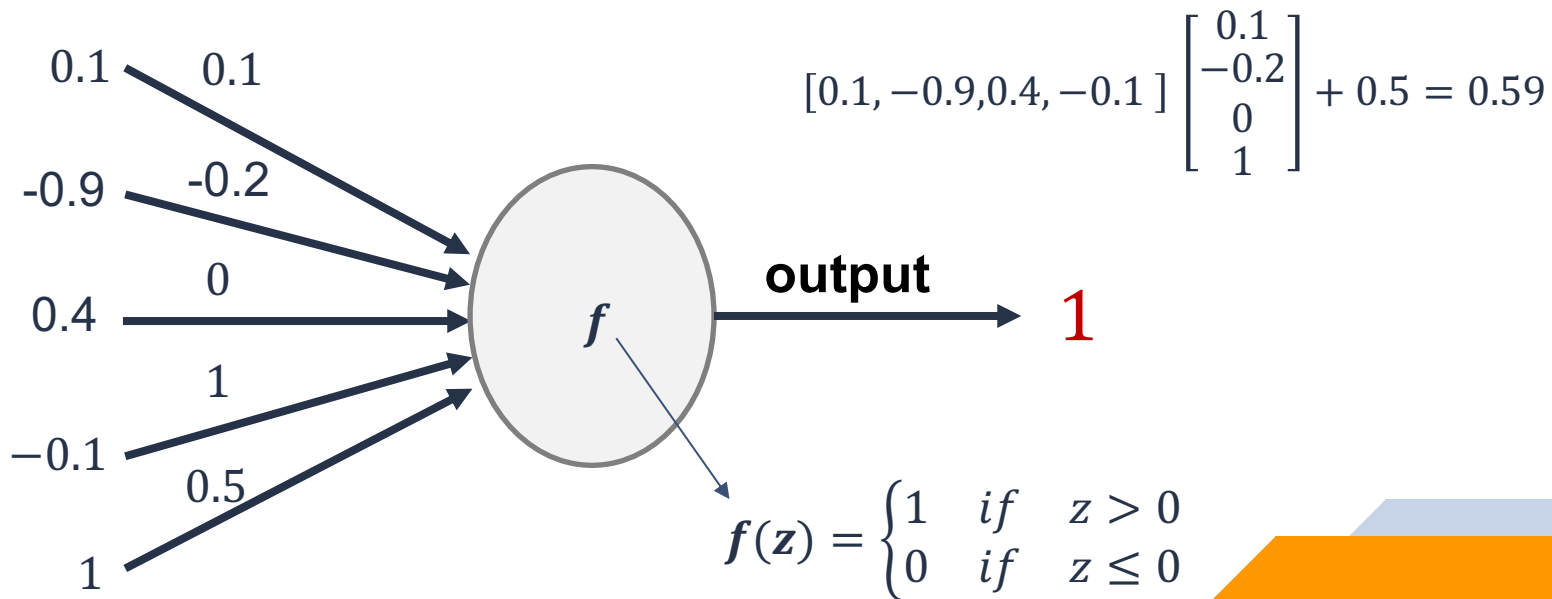
Exercise: write the forward pass in matrix notation and calculate the output of the perceptron for the given inputs and parameters values:





Perceptron: Definition

Exercise: write the forward pass in matrix notation and calculate the output of the perceptron for the given inputs and parameters values:



Python Programming Example



Perceptron.ipynb



Perceptron: Loss/Error

- ❑ By comparing the output with the expected value (ground truth) we can estimate perceptron's error/loss.
- ❑ There are multiple formulas for error/loss calculation.
- ❑ The simplest way is to compute the difference between the expected/target value and the actual output of the network:
$$y^j - \hat{y}^j$$



Perceptron: Learning

- ☐ Each example x^j from the training set is accompanied by a label y^j .
- ☐ The goal of the learning process is to construct a network that for an input value x^j it produces as an output a value that is close to y^j .
- ☐ Learning consists of modifying the weights so that we minimise the error/loss.

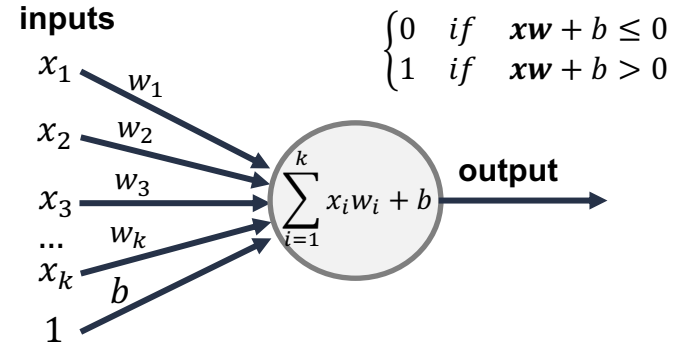
Perceptron: Learning with Simple Update Rule



Perceptron Learning: Simple Update Rule

□ Training Perceptron is easy!

Learning algorithm:



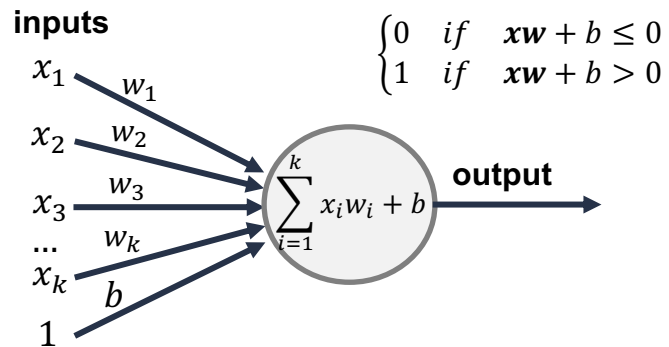


Perceptron Learning: Simple Update Rule

□ Training Perceptron is easy!

Learning algorithm:

Input: Training set $\{(x^1, y^1), \dots, (x^n, y^n)\}$





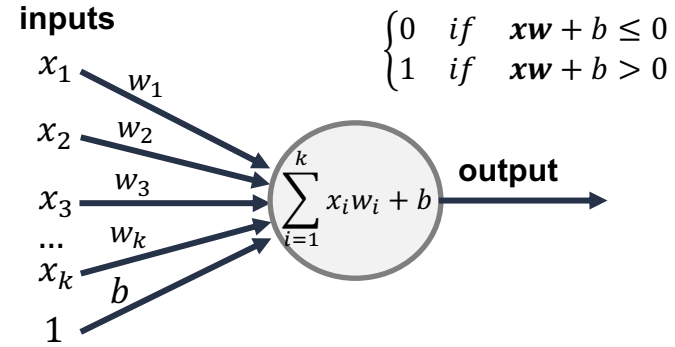
Perceptron Learning: Simple Update Rule

□ Training Perceptron is easy!

Learning algorithm:

Input: Training set $\{(x^1, y^1), \dots, (x^n, y^n)\}$

Initialise weights $w_1, \dots, w_k, b$ with random values





Perceptron Learning: Simple Update Rule

□ Training Perceptron is easy!

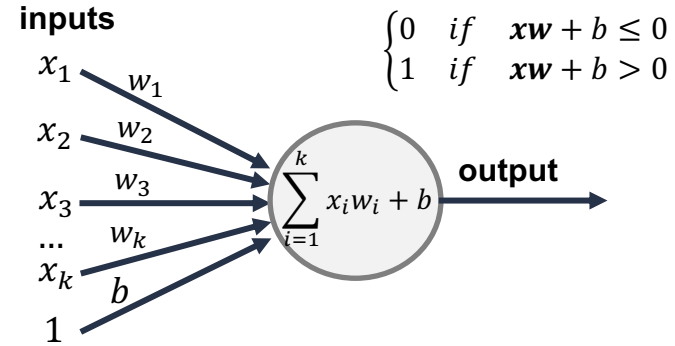
Learning algorithm:

Input: Training set $\{(x^1, y^1), \dots, (x^n, y^n)\}$

Initialise weights $w_1, \dots, w_k, b$ with random values

For each training item j :

(1) Present $x^j = \langle x_1^j, \dots, x_k^j \rangle$ as input to the perceptron





Perceptron Learning: Simple Update Rule

□ Training Perceptron is easy!

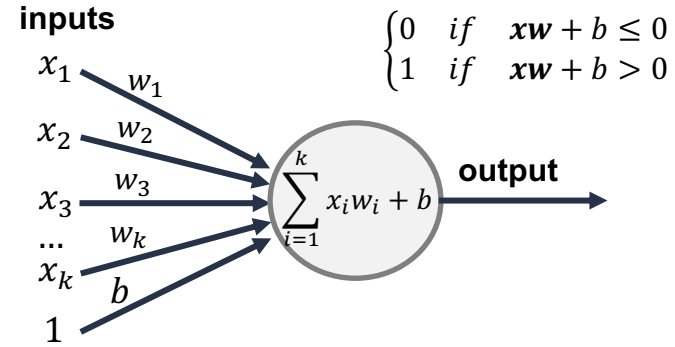
Learning algorithm:

Input: Training set $\{(x^1, y^1), \dots, (x^n, y^n)\}$

Initialise weights $w_1, \dots, w_k, b$ with random values

For each training item j :

- (1) Present $x^j = [x_1^j, \dots, x_k^j]$ as input to the perceptron
- (2) Compute the output $\hat{y}^j$





Perceptron Learning: Simple Update Rule

□ Training Perceptron is easy!

Learning algorithm:

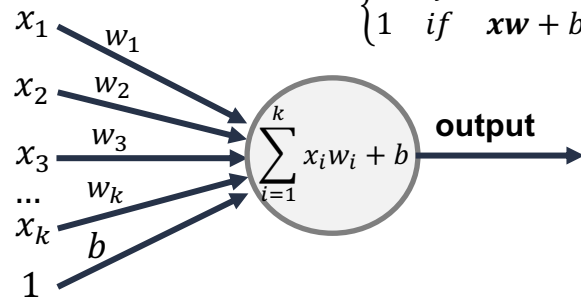
Input: Training set $\{(x^1, y^1), \dots, (x^n, y^n)\}$

Initialise weights $w_1, \dots, w_k, b$ with random values

For each training item j :

- (1) Present $x^j = [x_1^j, \dots, x_k^j]$ as input to the perceptron
- (2) Compute the output $\hat{y}^j$
- (3) Compute the difference between the expected value and the output: $y^j - \hat{y}^j$

inputs



$$\begin{cases} 0 & \text{if } \mathbf{xw} + b \leq 0 \\ 1 & \text{if } \mathbf{xw} + b > 0 \end{cases}$$

Desired	Output	Error
0	0	0
0	1	-1
1	0	1
1	1	0



Perceptron Learning: Simple Update Rule

□ Training Perceptron is easy!

Learning algorithm:

Input: Training set $\{(x^1, y^1), \dots, (x^n, y^n)\}$

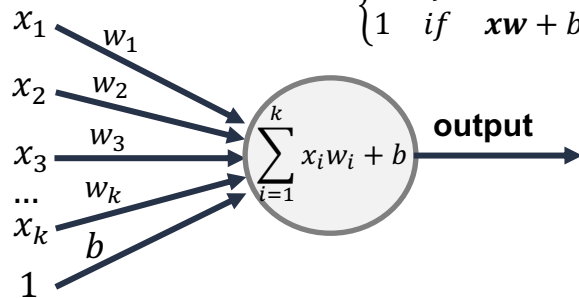
Initialise weights $w_1, \dots, w_k, b$ with random values

For each training item j :

- (1) Present $x^j = [x_1^j, \dots, x_k^j]$ as input to the perceptron
- (2) Compute the output $\hat{y}^j$
- (3) Compute the difference between the expected value and the output: $y^j - \hat{y}^j$
- (4) Adjust the weights according to the error:

$$w_i^{(next\ step)} = w_i + \eta(y^j - \hat{y}^j)x^i$$
$$b^{(next\ step)} = b + \eta(y^j - \hat{y}^j)$$

inputs



$$\begin{cases} 0 & \text{if } \mathbf{xw} + b \leq 0 \\ 1 & \text{if } \mathbf{xw} + b > 0 \end{cases}$$

Desired	Output	Error
0	0	0
0	1	-1
1	0	1
1	1	0



Perceptron Learning: Simple Update Rule

□ Training Perceptron is easy!

Learning algorithm:

Input: Training set $\{(x^1, y^1), \dots, (x^n, y^n)\}$, **learning rate η**

Initialise weights $w_1, \dots, w_k, b$ with random values

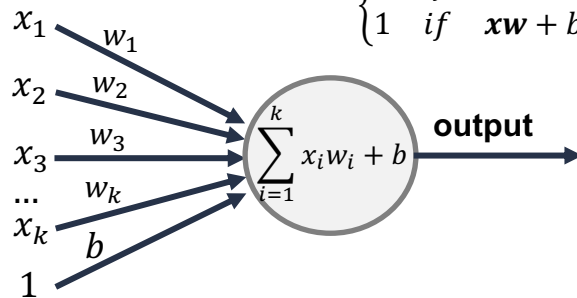
For each training item j :

- (1) Present $x^j = [x_1^j, \dots, x_k^j]$ as input to the perceptron
- (2) Compute the output $\hat{y}^j$
- (3) Compute the difference between the expected value and the output: $y^j - \hat{y}^j$
- (4) Adjust the weights according to the error:

$$w_i^{(next\ step)} = w_i + \eta(y^j - \hat{y}^j)x^i$$
$$b^{(next\ step)} = b + \eta(y^j - \hat{y}^j)$$

η (learning rate): smaller values mean slower changes to weights

inputs



Desired	Output	Error
0	0	0
0	1	-1
1	0	1
1	1	0



Perceptron Learning: Simple Update Rule

- Learning terminates when we reach a desired level of accuracy

Learning algorithm:

Input: Training set $\{(x^1, y^1), \dots, (x^n, y^n)\}$, learning rate η

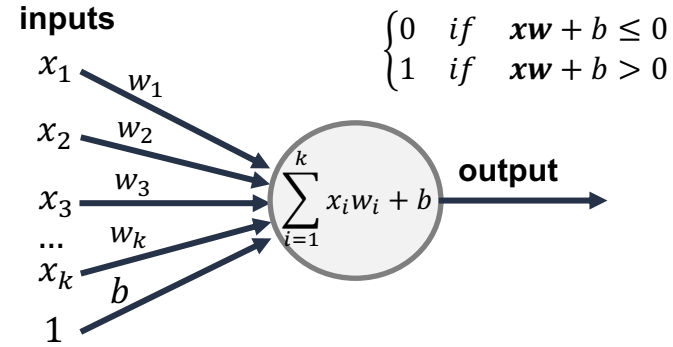
Initialise weights $w_1, \dots, w_k, b$ with random values

For each training item j :

- (1) Present $x^j = [x_1^j, \dots, x_k^j]$ as input to the perceptron
- (2) Compute the output $\hat{y}^j$
- (3) Compute the difference between the expected value and the output: $y^j - \hat{y}^j$
- (4) Adjust the weights according to the error:

$$w_i^{(next\ step)} = w_i + \eta(y^j - \hat{y}^j)x_i^j$$
$$b^{(next\ step)} = b + \eta(y^j - \hat{y}^j)$$

η (learning rate): smaller values mean slower changes to weights



Desired	Output	Error
0	0	0
0	1	-1
1	0	1
1	1	0

Python Programming Example



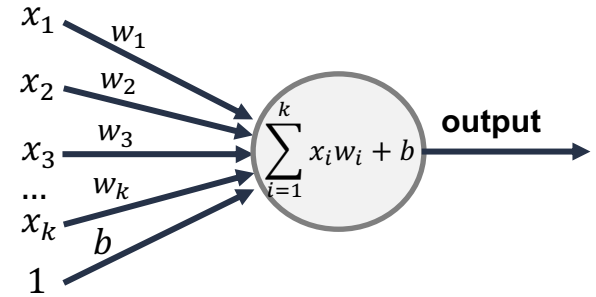
Perceptron.ipynb (Simple Update Rule)



Perceptron: Limitations

- ❑ Perceptron is very limited in its abilities.
- ❑ It can only solve linearly separable binary classification problems.

inputs



Coming Next:

Multilayer Perceptron